Table 1 SR operator variants: Instead of just asking the model to refine the solution, we also ask the model to find and analyze errors in the solution followed by the correct solution. Error analysis followed by solution generation leads to better performance for o3-mini but not for gemini-2.5-flash.

Model	Benchmark	Thinking Budget	**SR**	**SR**-*Error*
gemini-2.5-flash	AIME 2024	24576	88.75	87.71
gemini-2.5-flash	AIME 2025	24576	78.75	79.17
gpt-o3-mini	AIME 2024	24576	80.83	82.08
gpt-o3-mini	AIME 2025	24576	73.13	77.92

Parallel-Distill-Refine (PDR). We evaluate **PDR** under a fixed thinking budget B using three settings. These settings differ by number of rounds, number of parallel generations in each round, and selecting the k candidate solutions to carry forward via textual workspace: $g = [8]$, k=4; $g = [16, 8]$, k=4; and $g = [16, 8, 4]$, k=2. Here $g = [g_1, \ldots, g_r]$ specifies the number of parallel generations in each round, and $k \leq \min_d g_r$ is the number of candidates forwarded to the next round. For distillation (i.e., selecting the k candidates to carry forward), we compare: Random-k (uniform sampling per instance); Top-k (model-graded) where the same base model assigns a quality score to each candidate and we keep the top k per instance (we report both a shared rubric and a per-instance grading prompt); and global-summary that aggregates all candidates using a summarization prompt. Refinement, selection, and summarization prompts are detailed in §B.2.

Token-matched baselines. In Figure 5 we sweep the depth of **SR** to find the sequential budget B_{seq} at which it matches **PDR**. Holding the total token budget fixed at $B_{\text{total}} = 442$k (for **SR**, $B_{\text{total}} = B_{\text{seq}}$ since there is no parallelism), **PDR** attains the target accuracy with $B_{\text{seq}} = 172$k. To reach the same accuracy, **SR** is run for 17 rounds, consuming $B_{\text{seq}} \approx 442$k. Thus, at equal B_{total}, **PDR** achieves the same accuracy with a 2.57× smaller sequential budget by converting parallel compute into accuracy without lengthening per-call context.

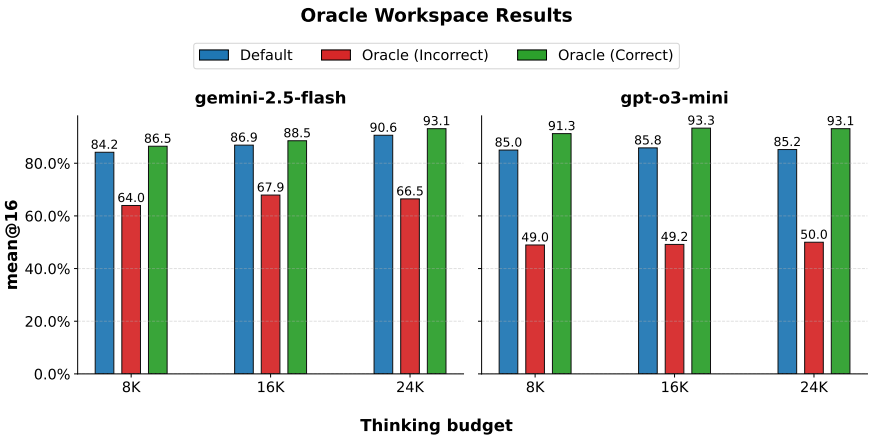

Figure 6 AIME 2024: Anchoring bias due to +ve and −ve examples: With **PDR** we compare three selection policies for the summary: Random-k, Oracle-Incorrect (all k candidates are incorrect), and Oracle-Correct (all k candidates are correct), evaluated on both gemini-2.5-flash and o3-mini. Across all thinking budgets, admitting only incorrect candidates into the summary yields a pronounced drop in accuracy, whereas admitting only correct candidates improves over the Random-k baseline. The degradation under Oracle-Incorrect is larger for o3-mini than for gemini-2.5-flash, indicating weaker self-verification in o3-mini.

Results. Figures 3 and 9 report accuracy on AIME 2024 and AIME 2025 under the same effective token budgets B_{seq}. We observe consistent gains when moving from long chain-of-thought to **SR**, which continue when moving from **SR** to **PDR**. For o3-mini at an effective budget of 49k tokens with a per-call thinking budget of 16k, accuracy improves from 76.9 (Long CoT) to 81.5 (**SR**) and 86.7 (**PDR**), an absolute improvement of +9.8 percentage points over Long CoT. gemini-2.5-flash shows smaller deltas from **SR** to **PDR** than o3-mini, suggesting stronger intrinsic self-verification in gemini-2.5-flash. AIME 2025 exhibits similar trends.

RQ2: Which distillation (i.e., summarization) strategy works best?

Table 2 Effect of distillation operator $\mathcal{D}$: We compare the effect on final performance by changing the distillation operator $\mathcal{D}$. Each table column reports accuracies on AIME 2024 / AIME 2025. We compare four choices: (i) *Global summary*: aggregate all candidates and synthesizes a single compact summary; (ii) *Per-sample top-k*: each downstream branch selects its own top-k candidates as the summary; (iii) *Shared top-k*: a single set of top-k candidates is shared as the summary across generations for next round; (iv) *Random-k*: each generation for next round receives k candidates sampled uniformly at random for the summary. Overall, global summary and per-sample top-k tend to perform best, with gains more pronounced at higher thinking budgets. For o3-mini on AIME 2025, global summary yields the largest improvement, suggesting strong summarization ability in o3-mini. We use $k = 2$ for these experiments.

Budget	gemini-2.5-flash				gpt-o3-mini			
	Global	PS top-k	Shared top-k	Random-k	Global	PS top-k	Shared top-k	Random-k
8192	83.13 / 66.88	83.75 / 71.88	84.17 / 70.21	83.33 / 66.67	86.04 / 82.92	84.79 / 76.04	85.00 / 76.67	82.50 / 71.25
16384	86.46 / 84.38	86.88 / 83.96	86.46 / 83.75	86.25 / 80.63	86.46 / 84.79	85.42 / 74.58	85.83 / 76.88	83.13 / 71.88
24576	88.75 / 87.71	90.63 / 85.00	87.71 / 85.42	88.13 / 79.58	85.42 / 83.54	85.21 / 77.92	85.00 / 75.42	82.29 / 72.08

Table 2 studies the distillation operator $\mathcal{D}$ in **PDR** under a (fixed number of rounds, number of generations in each round) setting $g = [16, 8, 4]$ with $k = 2$ candidates per round. Across datasets and base models, *per-sample top-k* and *global-summary* selection consistently outperform *shared top-k* and *random-k*, and the margin widens as the thinking budget B increases. The main exception is AIME 2025 with `o3-mini`, where global summary outperforms the alternatives. We hypothesize that `o3-mini`'s summarization is particularly effective at capturing cues from both correct and incorrect drafts, and these cues, when distilled, lead to stronger subsequent refinements.

RQ3: How does the verification abilities effect the inference time performance ?

Oracle PDR analysis. To understand the role of model verification within **PDR**, we intervene on the set of candidates admitted to the summary at each round. We use a three-round **PDR** with number of generations in each round as $[16, 8, 4]$ and top-k selection ($k = 2$), and compare: (i) **Random-k:** choose k candidates uniformly at random from the previous depth; (ii) **Oracle (Correct):** admit only correct candidates to the compact summary when available; (iii) **Oracle (Incorrect):** admit only incorrect candidates.

From Figures 6 and 8, we observe that injecting incorrect candidates (Oracle (Incorrect)) causes large drops for all models. The degradation is substantially larger for `o3-mini` than for `gemini-2.5-flash`, suggesting stronger self-verification and recovery in the latter. The same trend holds across AIME 2024 and AIME 2025.

We further provide a detailed analysis on the mechanics of **PDR** and self-verification requirements to improve downstream performance in Section E.

4.2 Operator-consistent RL Training

RQ4: Does operator-consistent training move the Pareto Frontier?

Building on the above, we present an operator consistent RL training strategy. This also addresses a train-test gap where models are not explicitly trained to perform **PDR**.

Training setup. We train an 8B dense model similar to Llama-3 style architecture (Dubey et al., 2024). For warm-start supervised fine-tuning (SFT), we use GPT-OSS 120B (Agarwal et al., 2025) to generate synthetic traces for math and code prompts sampled from Polaris-53K (An et al., 2025) and DeepCoder (Luo et al., 2025), respectively. We run SFT for 8B tokens (∼4 epochs). For RL training, we use the Polaris-53K dataset. Both SFT and RL datasets are decontaminated against AIME 2024/2025 (AoPS, 2025) and MATH-500 (Hendrycks et al., 2021). Further details and hyper-parameters are detailed in Section B.3.

Baseline RL As described in Section 3.3, we use the CISPO objective for RL post-training (Li et al., 2025). We set $\epsilon^- = 0.0$ and $\epsilon^+ = 5.0$, and remove "zero-variance" prompts from a given batch (Seed et al., 2025). We use forced interruptions (Hong et al., 2025; Yang et al., 2025) to control generation length from exploding after a thinking budget of $16,384$ tokens. We additionally keep a buffer of 2048 tokens for the final solution, thus keeping a maximum generation length of 18432. 32 generations are sampled per prompt with a batch size of 32, resulting in a global batch size of 1024 generations per gradient step. Following (Liu et al., 2025b), we use a mini-batch size of 256 and perform 4 gradient updates per rollout step.

Table 3 Operator RL results: Comparison of RL training objectives on AIME 2024/2025 at matched sequential budget $B_{\text{seq}} = 65{,}536$ tokens using a dense 8B model. Mixing standard RL with operator-consistent RL (Op-RL) yields consistent gains for iterative inference operators such as **PDR** while preserving performance on the Long CoT baseline. Op-RL can also be applied as a continual RL to the existing baseline RL checkpoint.

Model	AIME 2024		AIME 2025	
	Long CoT	**PDR**	Long CoT	**PDR**
8B SFT Policy	47.50	62.92	35.00	47.50
8B Baseline RL	67.50	75.83	59.58	65.83
8B **PDR** RL	69.58	79.17	57.50	67.50
8B Continual **PDR** RL	70.00	80.83	61.25	70.42

Operator-consistent RL with PDR. For training, we use the **PDR** operator with configuration (4 parallel generations, 1 round) $g = [4]; k = 2$, and use the training objective described in Equation (11) and make two changes to the baseline RL method above: (i) increasing the input prompt length from 2048 tokens to 10240 to allow for the compact workspace to be a part of the input, and (ii) mixing the standard RL and operator RL batches in the dataloader, keeping all other design choices the same. This setup allows to scale inference compute within the RL training.

Results. Table 3 summarizes the main results. The resulting model from each RL objective is evaluated for Long CoT generation and **PDR**. **PDR** RL improves over the baseline by +3.34 points on AIME 2024 and +1.67 points on AIME 2025. With continual updates starting from a baseline RL checkpoint, additional **PDR** RL yields larger gains of +5.00 and +4.59 percentage points on AIME 2024 and AIME 2025, respectively. Additionally, we also observe marginal gains on Long CoT generations with **PDR** RL training. These results indicate that training with operator-consistent RL objectives reduces the mismatch between training and deployment, converting extra compute into accuracy without increasing the per-call sequential budget.

5 Conclusion

In this paper, we initiate the exploration of a broader design space around "long CoT." We study two operators in this design space, **SR** and **PDR** which give better accuracy compared to standard long CoT, while offering the benefit of smaller context size. Empirically, compact-memory iteration outperforms long-trace baselines at matched B_{seq}. **PDR** yields the largest gains (e.g., +11% on AIME 2024 and +9% on AIME 2025), showing that evidence accumulation via bounded summaries can substitute for long reasoning traces while holding latency fixed. Beyond inference orchestration, making sure that training is consistent with inference using an *operator-consistent RL* objective further improves performance (e.g., $\sim 5\%$ on AIME 2024 and AIME 2025), suggesting that models can learn the meta-skills that make iteration effective. Iterative reasoning improves when diversity, verification, and refinement become reliably good; by measuring and training these micro-skills directly, we can accelerate the gains of improvement operators under fixed latency budgets.

Promising future directions include learning to improve the synthesis operator $\mathcal{D}$ (trainable summaries), adaptive round and fan-out schedules conditioned on uncertainty (adaptive $top - k$), budget-aware controllers for allocating test-time compute, and tighter integration with verifiers and tool use. We also see value in scaling studies and cross-domain evaluations (reasoning, coding, and planning) to map when short-context iteration most benefits accuracy and latency.

6 Acknowledgments

This work includes contributions from S.G. during his time at Meta. We also thank Xuewei Wang, Devvrit Khatri, and Alan Schelten for helpful discussions, and Jenya Lee and Abhinav Jauhri for infrastructure and compute support.